University of Washington

XPL Compiler

by

Geoffrey Leach

Technical Report No. 72-12-06

Computer Science Group

University of Washington
Seattle, Washington 98195

December 1972

ACKNOWLEDGEMENT

The largest part of the work required to achieve the final version of the XPL compiler described herein was the coding of the initial version in Sigma 5 assembly language and the implementation decisions that were required to account for the differences in architecture. This work was done by: Gregory Andrews, Mary-Beth Felcyn, John Jensen, Geoffrey Leach, Linda Meinke and Gain Wong, graduate students in the Computer Science Group at the University of Washington. The transliteration of the assembly language into XPL was done by John Jensen, Geoffrey Leach and Linda Meinke.

All those who worked on the project profited from the advice of Professor Hellmut Golde, Director of the Computer Science Laboratory, and Professor Jean-Loup Baer, both of whom provided advice and counsel to the project from the beginning.

The work described by this report was to a large degree made possible by the original work on XPL of William B. McKeeman, James J. Horning and David B. Wortman.

The report was proofread by Susan Leach, who also suggested many stylistic changes.

This work was supported in part by National Science Foundation Grant GJ 28781.

Table of Contents

1. Introduction 1
2. History of the Project 2
3. Program Structure 4
4. Data Structure 8
5. Code Generation 11
6. Conclusions 13

Appendices

1. Narrative Description 14
2. The Run-Time Support Module 40
3. Extensions to XPL 43
4. Language Differences: XPL/360 and XPL/I5 45
5. Interpretation of XPL Listings 46
6. Object Language Mnemonics 30

References 51

1. INTRODUCTION

This report documents the implementation of a compiler for the language XPL on the Xerox Data Systems Sigma 5 computer [2], under the Realtime Batch Monitor (RBM) [4]. The source and inspiration for this work was the work by McKeeman, Horning and Wortman, as reported in their book, A Compiler Generator [1]. This document is intended to be a supplement, for the particular case of the Sigma 5 implementation. In many instances, the reader will be referred to [1], particularly in those cases where we have adopted concepts or program without substantial alteration. References to "the original XCOM" refer to the version written for the IBM System/360 and documented in [1].

XCOM, the compiler for XPL, is the first such program to have received such wide and effective distribution. One point of interest is how the present work reflects upon the effectiveness of this mode of communication of programs.

The Sigma series of computers is somewhat similar to System/360, for which the original XCOM was designed. Another point of interest is the way in which the differences in architecture between the Sigma 5 and S/360 manifest themselves in the logic of the code generation sections of the compiler.

Finally, there is the XCOM program itself. Among the principal goals of McKeeman et. al.[1, p.191] were: "To equip the reader either (1) to change the present version of XCOM or to modify the language XPL to better suit his needs or (2) to duplicate our efforts by producing a similar compiler for another machine." In writing this report, we have also adopted these goals. It is our hope that as a result of our efforts it will be easier still for them to be achieved.

The chapters of this report contain a general discussion of the problems encountered and the decisions made in the course of the implementation of XPL on the Sigma 5 computer. They should be of interest to the reader who has a knowledge of XPL and some familiarity with the architecture of the Sigma 5.

Appendix 1 contains a narrative description of the compiler, by procedures. It is intended to convey detailed information concerning the logic and techniques used, as an aid to maintenance and modification.

Detailed familiarity with the architecture of the Sigma 5 is assumed. Familiarity with the REM operating system is only required for the discussion of the Run-Time Support (RTS) module, in Appendix 2.

The language XPL is not discussed in this report. The formal definition of its syntax may be found in [1, ch.6]. The language that has been implemented is essentially that described. Such differences as exist are summarized in Appendix 4.

2. HISTORY OF THE PROJECT

The XPL project was initiated in May 1971 by a group of computer science graduate students. Its overall aims were:

1. To provide the members of the group with some experience in the implementation of a substantial piece of software.
2. To make a worthwhile addition to the facilities available on the Sigma 5 computer in the Computer Science Teaching Laboratory.

A number of alternatives were considered, but it was felt that, at that time, the most useful software would be a language processor that would facilitate further research. The alternatives considered were XPL, EULER and PL/360. XPL was finally chosen because:

1. It was part of a well documented translator writing system and therefore might be expected to motivate efforts to implement other languages.
2. The existence of a compiler for XPL for a similar machine (System/360) would reduce the amount of work required.

The facilities available for the implementation were:

1. The Sigma 5 computer [2] with 24576 words of main memory, 17000 of which were available for program storage.
2. A Macro-assembly language, MACROSYMBOL [3].
3. A FORTRAN IV compiler.

At the outset, it was feared that the limited memory would make implementation difficult and might even prevent successful bootstrapping to an XPL version of the compiler. This consideration and the fact that there is no particular advantage to writing in FORTRAN, directed the choice of assembly

language to implement the bootstrap compiler.

The technique used to implement the bootstrap version was to code macros that were as close as possible to the various XPL constructs. Thus, there was a DECLARE macro, a DO_WHILE macro, and so on. In fact, there were macros for almost every XPL construct. Unfortunately, the facilities of the MACROSYMBOL language are such as to require the generation of large numbers of extraneous labels to connect the parts of conditional and iterative statements. For instance, the IF macro required, as a parameter, the label on the THEN macro. This led to a rather messy program. There was no way to determine the declared type of a variable automatically. In fact, most errors in the code arose from the accessing of variables with incorrect storage type. It might also be mentioned that the resultant code was not particularly efficient. The use of macros often generated unnecessary data or instructions. It was felt that convenience was the more important consideration and that if storage problems should arise later, recoding would be possible.

The absence of any automatic block structuring facility in the MACROSYMBOL language caused serious problems of data management. The large number of global variables and macros ruled out the use of separately assembled subroutines. To avoid duplicate declaration it was necessary either to use the LOCAL directive provided by MACROSYMBOL or to adopt some artifice to make names unique in local context. The latter was choosen. Local names had a routine number appended. Thus, the declaration of "I" in routine 24 became "I:2400". This served to clutter the program text and in any case did not preclude the accidental use of the wrong variable. In retrospect, it was the wrong choice.

In addition to the self-documentation provided by the resemblance of the macros to the original XPL, the actual XPL source statements were interspersed, as comments, with the assembly language code. Thus a person could, with some concentration, read the compiler "in XPL".

The consensus among the implementors of the MACROSYMBOL XCOM was that the approach was successful. Certainly this is true in terms of the effort required to produce a version that was sufficiently error free to compile the XPL version of the compiler, which was 1 man-month to design and implement the macros and 4 man-months to implement and debug the compiler.

language to implement the bootstrap compiler.

The technique used to implement the bootstrap version was to code macros that were as close as possible to the various XPL constructs. Thus, there was a DECLARE macro, a DO_WHILE macro, and so on. In fact, there were macros for almost every XPL construct. Unfortunately, the facilities of the MACROSYMBOL language are such as to require the generation of large numbers of extraneous labels to connect the parts of conditional and iterative statements. For instance, the IF macro required, as a parameter, the label on the THEN macro. This led to a rather messy program. There was no way to determine the declared type of a variable automatically. In fact, most errors in the code arose from the accessing of variables with incorrect storage type. It might also be mentioned that the resultant code was not particularly efficient. The use of macros often generated unnecessary data or instructions. It was felt that convenience was the more important consideration and that if storage problems should arise later, recoding would be possible.

The absence of any automatic block structuring facility in the MACROSYMBOL language caused serious problems of data management. The large number of global variables and macros ruled out the use of separately assembled subroutines. To avoid duplicate declaration it was necessary either to use the LOCAL directive provided by MACROSYMBOL or to adopt some artifice to make names unique in local context. The latter was choosen. Local names had a routine number appended. Thus, the declaration of "I" in routine 24 became "I:2400". This served to clutter the program text and in any case did not preclude the accidental use of the wroog variable. In retrospect, it was the wrong choice.

In addition to the self-documentation provided by the resemblance of the macros to the original XPL, the actual XPL source statements were interspersed, as comments, with the assembly language code. Thus a person could, with some concentration, read the compiler "in XPL".

The consensus among the implementors of the MACROSYMBOL XCOM was that the approach was successful. Certainly this is true in terms of the effort required to produce a version that was sufficiently error free to compile the XPL version of the compiler, which was 1 man-month to design and implement the macros and 4 man-months to implement and debug the compiler.

In spite of this, it was not felt that the MACROSYMBOL version was useful in any role other than that of a steppingstone. The detection of errors was an order of magnitude more difficult than in the XPL version. Substantial modification of the code for reasons of efficiency or elegance was considered to be out of the question. By way of comparison, the assembly of the first version required fifteen minutes of elapsed time, while compilation of the XPL version with the MACROSYMBOL version required two and one half minutes.

Translation of the MACROSYMBOL code to XPL required only about one man-month. "Translation" amounted to the extraction of the XPL comments that had been inserted in the code. The majority of the errors in this version resulted from corrections that had been made to the MACROSYMBOL, but were not reflected in the comments.

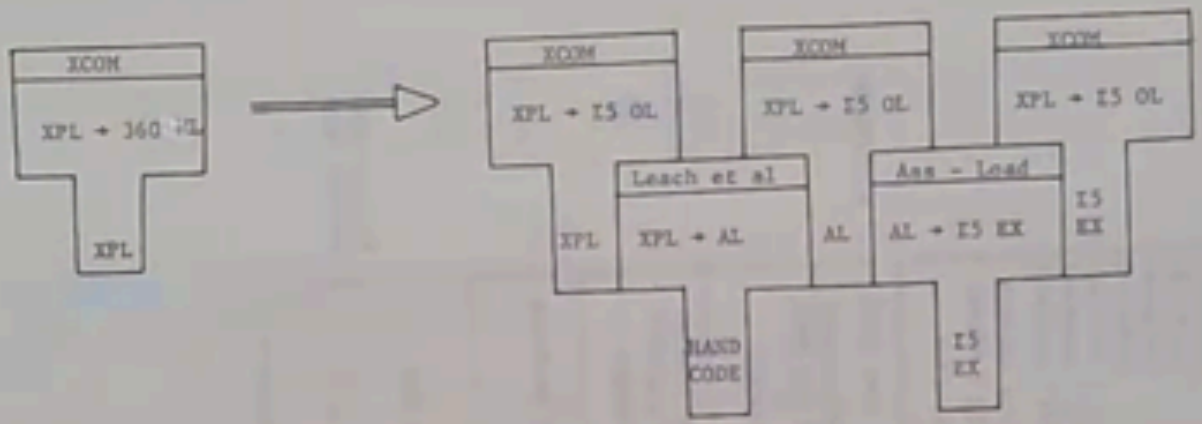
Figures 1 and 2 summarize in "T-diagram" notation the steps required to implement the final, self-compiling, version of XCOM. These may be compared with similar diagrams for the S/360 project [1, front endpapers and pp.227-229]. It will be observed that there is no sub-monitor. The functions of the sub-monitor are incorporated into the RTS module, coded in MACROSYMBOL, which is loaded with every XPL program by the REM overlay loader [4].

3. PROGRAM STRUCTURE

This section deals with the structuring of XPL programs and the allocation of main memory among the various components.

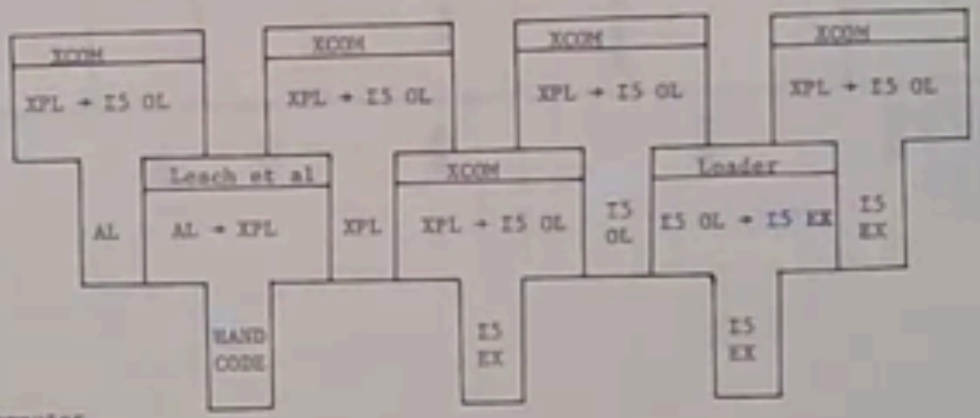
Figure 3 summarizes the allocation of main memory to the various parts of an XPL program. As the XPL compiler is itself an XPL program, the figure applies to it as well. The size of each area of the compiler is given at the right of the diagram. The first four sections, code, data, descriptor and string, are produced by the compiler as a single load module. The fifth section, RTS, is the second load module and is added by the loader. All of main memory that remains is placed in the sixth section.

The code section contains the compiled instructions that correspond to the XPL source program. These are not self-modifying. The only possible



First Bootstrap: XCOM/360 to MACROSYMBOL

Figure 1



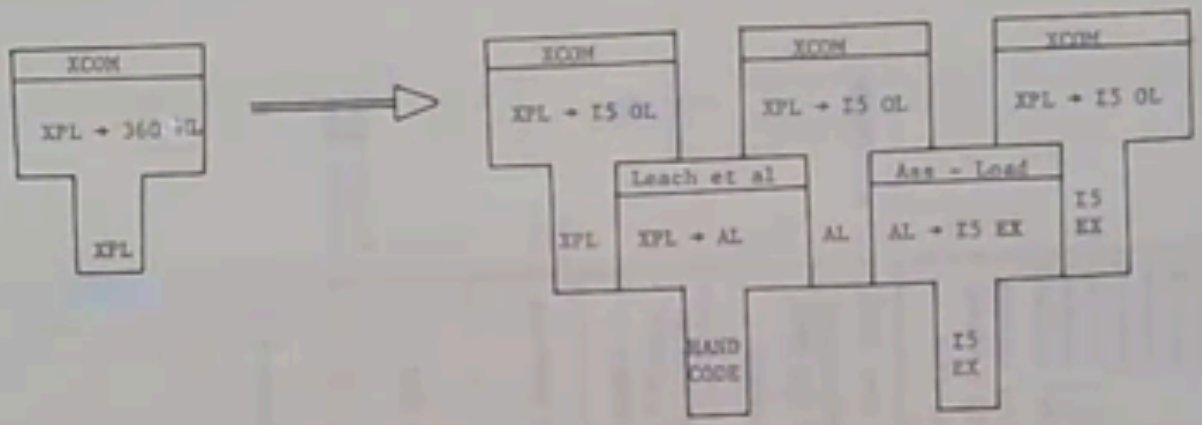
LEGEND

XPL	The language
XCOM	The compiler for XPL
AL	The assembly language
EX	Sigma 5 execution
OL	RBM Object Language
I5	Xerox Data Systems Sigma 5 Computer
RBM	Remote Batch Monitor operating system
ML	Machine language (360)

Second Bootstrap: MACROSYMBOL to XPL

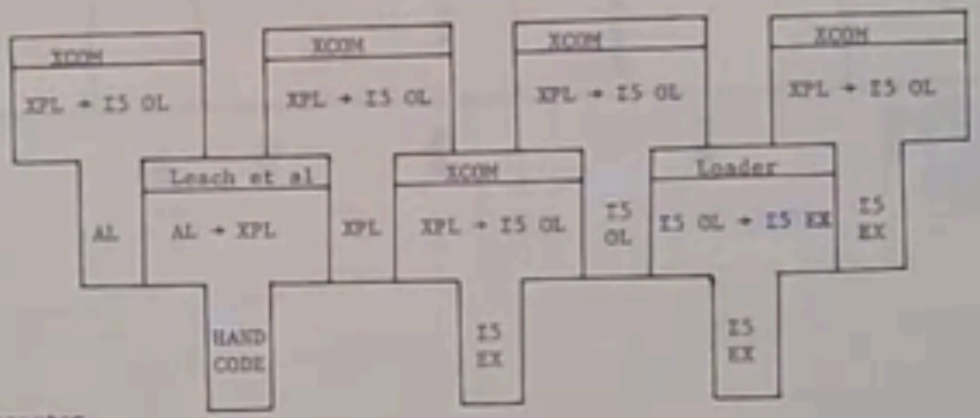
Figure 2

70-12-72



First Bootstrap: XCOM/360 to MACROSYMBOL

Figure 1



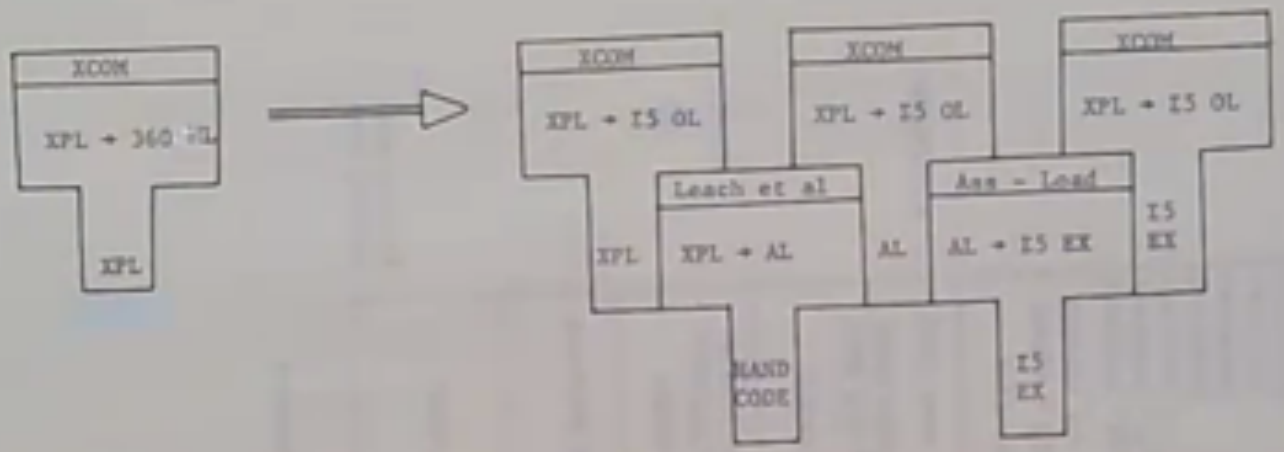
LEGEND

XPL	The language
XCOM	The compiler for XPL
AL	The assembly language
EX	Sigma 5 execution
OL	RBM Object Language
I5	Xerox Data Systems Sigma 5 Computer
RBM	Remote Batch Monitor operating system
ML	Machine language (360)

Second Bootstrap: MACROSYMBOL to XPL

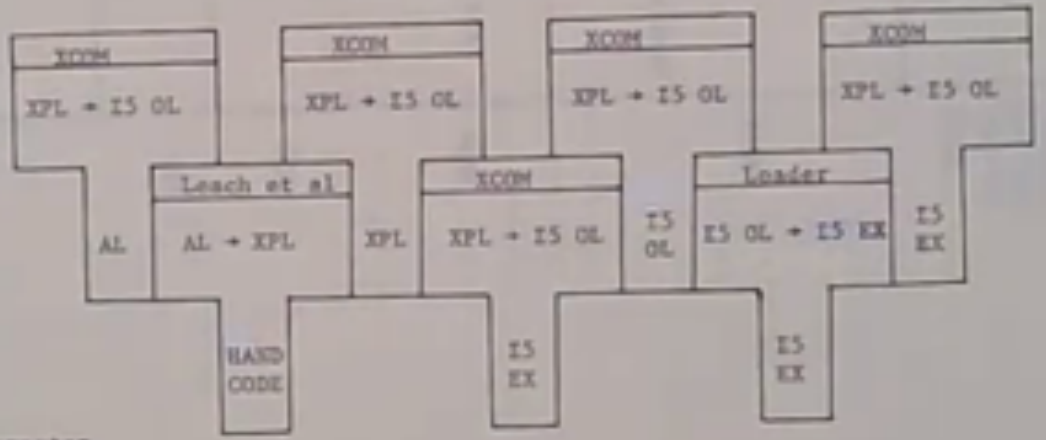
Figure 2

70-12-02



First Bootstrap: XCOM/360 to MACROSYMBOL

Figure 1



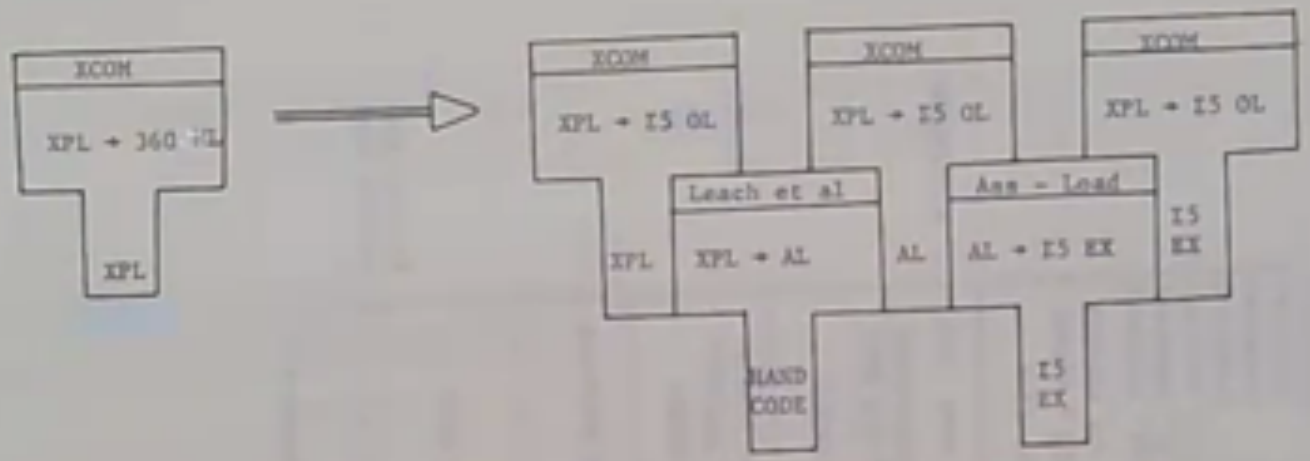
LEGEND

XPL	The language
XCOM	The compiler for XPL
AL	The assembly language
EX	Sigma 5 execution
OL	RBM Object Language
I5	Xerox Data Systems Sigma 5 Computer
RBM	Remote Batch Monitor operating system
ML	Machine language (360)

Second Bootstrap: MACROSYMBOL to XPL

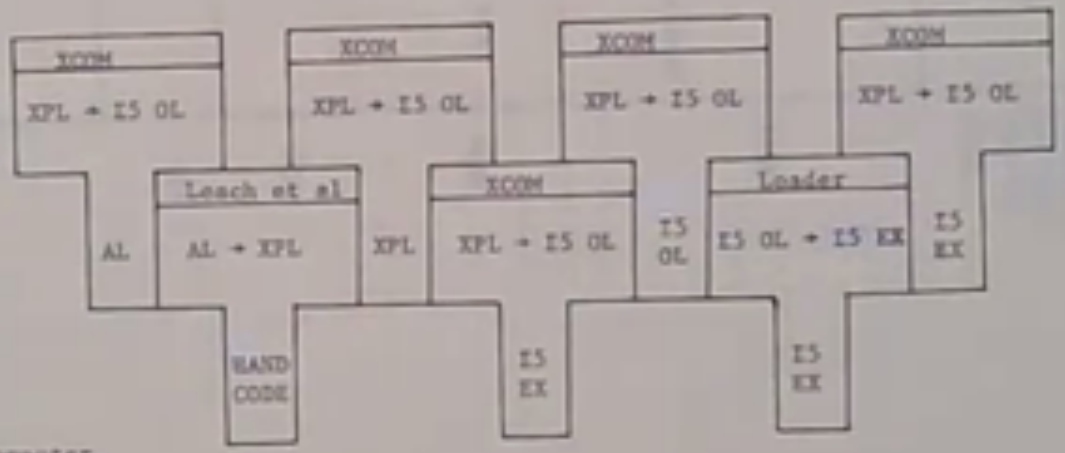
Figure 2

20-21-22



First Bootstrap: XCOM/360 to MACROSYMBOL

Figure 1



LEGEND

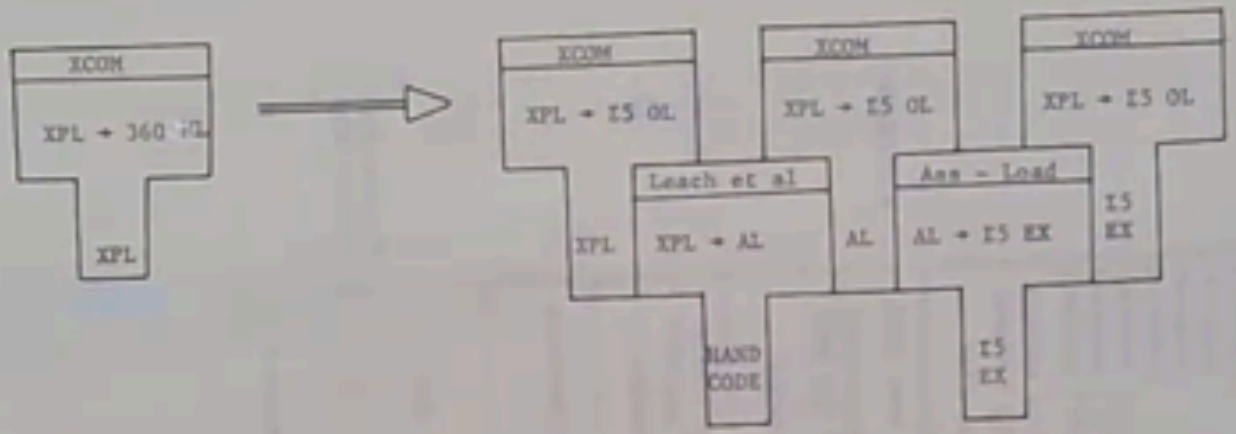
XPL	The language
XCOM	The compiler for XPL
AL	The assembly language
EX	Sigma 5 execution
OL	RBM Object Language
I5	Xerox Data Systems Sigma 5 Computer
RBM	Remote Batch Monitor operating system
ML	Machine language (360)

Second Bootstrap: MACROSYMBOL to XPL

Figure 2

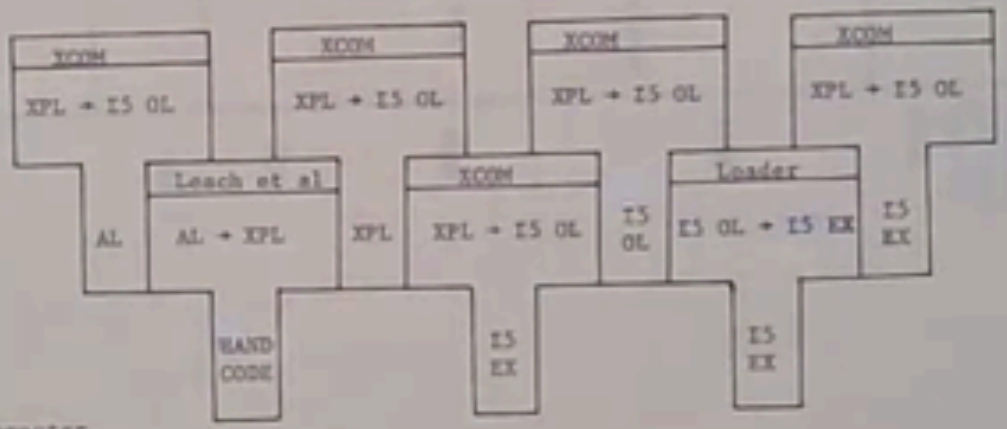
20-21-22





First Bootstrap: XCOM/360 to MACROSYMBOL

Figure 1



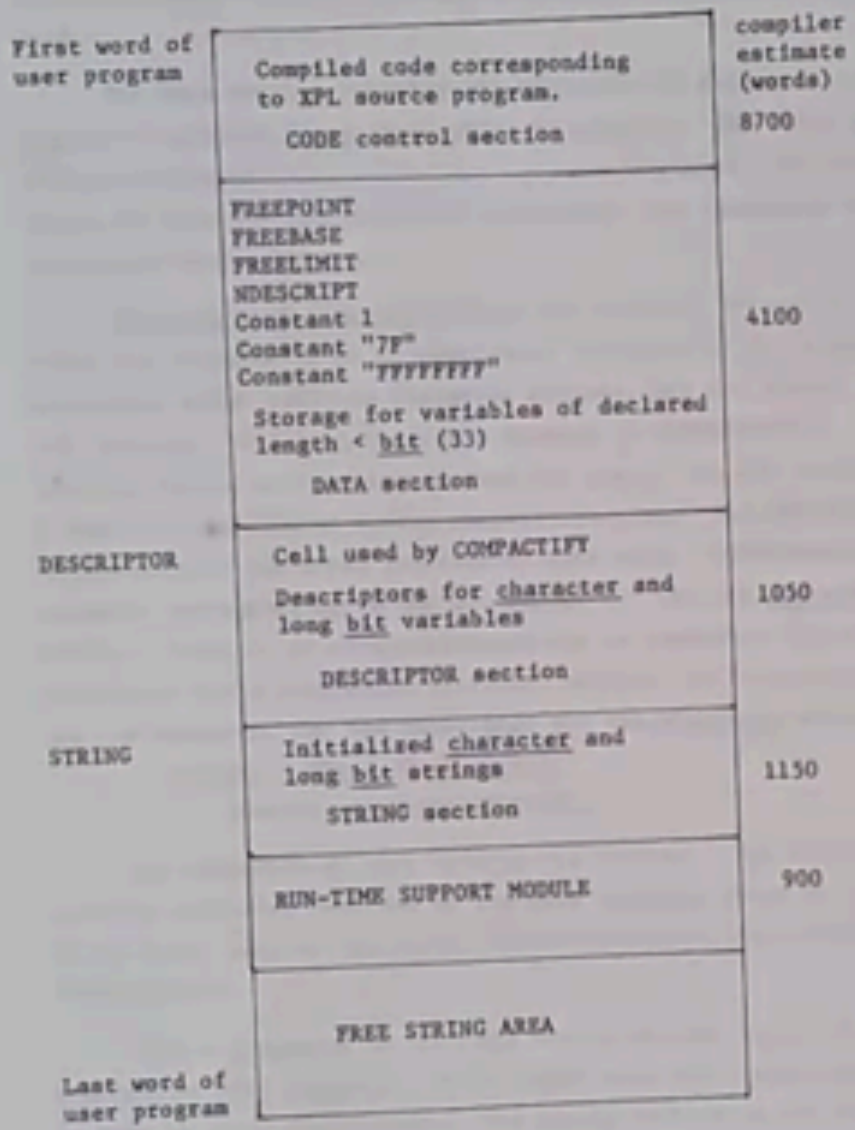
LEGEND

XPL	The language
XCOM	The compiler for XPL
AL	The assembly language
EX	Sigma 5 execution
OL	RBM Object Language
I5	Xerox Data Systems Sigma 5 Computer
RBM	Remote Batch Monitor operating system
ML	Machine language (360)

Second Bootstrap: MACROSYMBOL to XPL

Figure 2

70-12-72



Storage allocation for XPL programs

Figure 3

72-12-04

22-12-06

exception to this is that the user might elect to do so, through the builtin function INLINE. The memory write protection facility of the Sigma 5 is not used.

The data section contains all variables of declared size less than bit(33), including fixed type, which is bit(32). The first seven locations have special uses and are allocated by the compiler. The first four of these are used for the variables indicated. The remaining three contain frequently used constants.

Character and long bit strings are accessed through a descriptor. These are required to be in contiguous locations by the compactification algorithm, which collects discarded strings, and are stored in the descriptor section. The length of this section is NDESCRIPT+1. The first location in the section is reserved for use by the RTS routines, to save a temporary descriptor during compactification. The DESCRIPTOR variable corresponds to the first location of this area. DESCRIPTOR(J) is a fixed variable corresponding to the descriptor for the Jth character or long bit string. There is no straightforward way to determine the subscript of the descriptor for a particular variable, however the following example shows how the subscript for the descriptor for the character variable S is computed.

```
declare J fixed, S character;
J = ADDR(S) - ADDR(DESCRIPTOR);
```

The semantics of this code are as follows. The difference of the absolute addresses returned by the ADDR function gives an address relative to the first word of the array of descriptors or the subscript, as in DESCRIPTOR(J).

When a character or long bit string appears in an XPL program (as a constant or with initial), it is loaded into the string section. No attempt is made to avoid duplication. The string section is not subject to compactification, so these strings remain, even when they are no longer the object of any descriptor. The string section may be accessed as a byte array, using the variable STRING. For example, STRING(FREEPOINT) is the first available byte in the Free String Area (FSA).

The RTS module contains the code for the input/output interface between the XPL constructs and the RBM system, binary to decimal and hexidecimal conversion and the COMPACTIFY procedure, which is not compiled

from XPL, as is the case in the original system [1, pp.211-213]. This module is written in MACROSYMBOL and is loaded with every XPL program. It is discussed in Appendix 2.

The FSA contains volatile strings. Space is assigned as required by INPUT, FILE, number conversion and concatenation. When FREEPOINT moves past FREELIMIT, COMPACTIFY is invoked automatically. This process is described in detail [1, pp. 211-213]. The initialization of the pointers for the FSA, FREEPOINT, FREEBASE, and FREELIMIT take place at the beginning of each XPL program, so that maximum space is assigned to the FSA. As compactifications are the principal limitation to the speed of string operations, this helps to maximize processing speed.

4. DATA STRUCTURE

This section deals with the particulars of storage allocation for the XPL data types. It is organized according to the mode of storage addressing that is compiled for the data type.

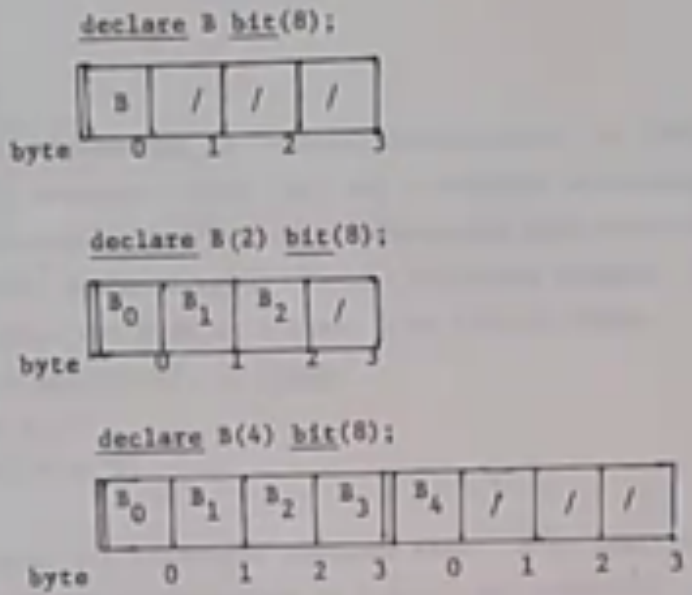
bit(1) - bit(8)

These types are compiled with byte-addressing instructions. As the Sigma 5 addressing mechanism requires that byte addressing be done with a word address and a byte index, only the leftmost byte can be accessed without specifying an index register. For this reason, simple byte variables are allocated in the leftmost byte of the word, with the remaining bytes left unused. Byte arrays are allocated in ascending byte locations, beginning with a word boundary. Figure 4 illustrates some of the possibilities. It would be possible to pack byte variables four to a word, but the savings in data storage would be more than offset by the code generated to index every access of the three rightmost bytes.

bit(9) - bit(16)

These types are compiled with halfword-addressing instructions. Allocation and addressing is handled in the same way as the byte addressing types described above. Figure 5 illustrates two cases.

72-12-04

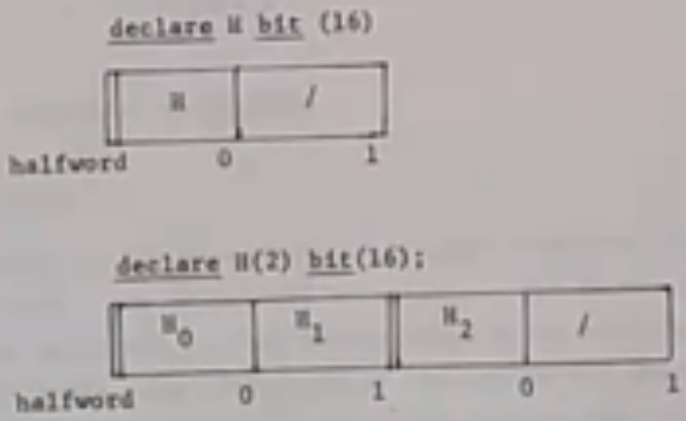


/ : not used

|| : word boundary

Storage allocation for byte variables

Figure 4



|| : word boundary

/ : not used

Storage allocation for halfword variables

Figure 5

72-12-04

bit(17) - bit(32)

These types are compiled as fullword instructions, so there is no storage waste. It is worth noting that the allocation mechanism permits variables to be accessed using a storage addressing mode other than the one implied by their declared type. In the following example, the contents of F are output one byte at a time, from left to right.

```
declare B bit(8), (F, J) fixed;
do J = 4 to 7;
   OUTPUT = B(J);
end;
```

Figure 6 illustrates the principle involved here. Note that the scheme acts differently if the declarations of F and J are reversed. Quite obviously, the technique is open to the criticism of being implementation dependent. It is suggested that this is not per se important, but that we should be aware of the dependency.

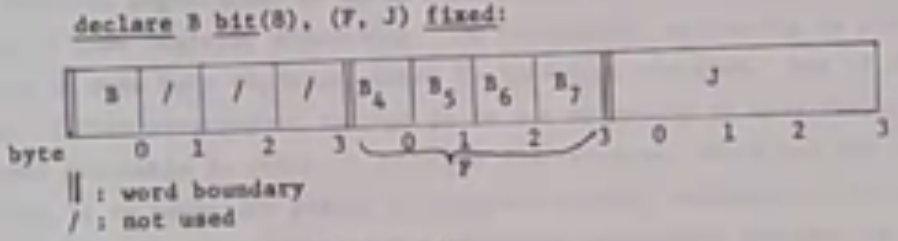


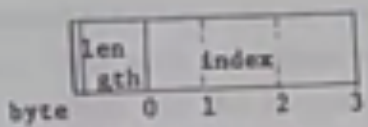
Figure 6

fixed

This type is identical to bit(32).

character, bit(n), n>32

As was mentioned in the discussion of program structure, character and long bit data have an associated descriptor. The format of this descriptor is shown in figure 7. The length field gives true length and has range 1 - 225. Hence, the limitation of strings to 255 characters. String values are packed, so there is no storage loss in the case of strings whose length is not a multiple of four. The descriptor representation of the null string, '', is zero length.



‖ : word boundary

descriptor format

Figure 7

5. CODE GENERATION

The code generation strategy is essentially that described by McKeeman [1, pp. 219 - 225]. There have been numerous small changes in detail, as would be expected with the differences in architecture. The code that is generated for specific syntatic constructs in the language may be determined with the use of listing options. Appendix 5 illustrates the output produced and how it is interpreted.

The most significant of these differences for code generation is the absence of base register addressing. On System/360, addressing is performed relative to some base register, which is loaded at run-time. One of the results of this arrangement is that it is not necessary to have absolute addresses available in order to generate instructions. This has the drawback that it is necessary either to allocate certain registers to the role of base registers, thus reducing the statement processing ability, or to generate extra code for the loading of base registers immediately prior to use.

On the Sigma 5, there are two solutions possible to the problem posed by the absence of base registers, if one wishes to maintain similarity to the original XCOM: (1) Simulation of the base registers with index registers and (2) use of a second pass to provide the correct absolute address for instructions requiring it.

The first solution requires that all instructions accessing, say, the data area be indexed on a register containing the base of the data area. In fact, access to the string area is handled in exactly this way. However, this solution fails if an access requires additional indexing. Then, we are forced to fall back on run-time computation of addresses, which would considerably increase the size of any program that uses arrays.

22-12-06

The second solution, the one adopted, is to emit absolute addressing instructions. This must be accomplished through a relocation phase in the compilation process, for the necessary information is not always available until the entire program has been scanned. Again, two possibilities are open, although, as before, one suffers from numerous drawbacks.

If one insists that the result of the compiler be an absolute program, then it is necessary to maintain a list of locations that need to be relocated. Upon completion of the code emission, the proper base address is added to each of these. This requires storage for the lists of fixups and would result in a decrease in compiler speed for fixed total memory, as speed is in part dependent on the size of the FSA.

Furthermore, a compiler that produces absolute code suffers in that it is difficult for it to link to independently compiled modules, whether in the source language or some other. The result of this failing is apparent in the original XCOM. There, one finds sections of code devoted to the task of producing code to be included in each program for certain support tasks. It is also necessary to compile a copy of the COMPACTIFY routine at the beginning of every program and this special case must be taken into consideration at several places. In general, programs produced by such a compiler tend to be more isolated from the environment in which they are used than those passing through a relocating loader.

The use of a relocating loader, essentially a second pass in the compilation, permits these failings to be overcome. There is no necessity for elaborate fixups to be made within the compiler. For the most part, the compiled code is forgotten by the compiler once it is emitted. Routines that are common to every program may exist as a separate module and loaded with each. As a result, the compiler is smaller and faster.

We have no specific measurements of the time-space tradeoffs. It is probable that the loader requires more time than a second pass within the compiler, as it handles a more general case than an XPL program, but it is certain that the compiler is faster as a result of the omission of the fixup lists.

No facility for the separate compilation of XPL routines has been introduced, as this would require an extension to the syntax. Such an extension is discussed in Appendix 3. A limited facility for the access of external

routines is provided. Its use is discussed in Appendix 2.

6. CONCLUSIONS

It is of interest to evaluate the results of the project in two respects. (1) How does it reflect upon the publication of software, as an aid to others' implementations [1, p. 191]? (2) How successful was the bootstrapping method?

The existence of the present report is an affirmative answer to the first question. Even considering the fact that the architecture of Systems/360 and the Sigma series are quite similar, we have succeeded in producing a working compiler with only a very moderate outlay in man and machine time. It is certain that we would not have accomplished this without the base of a working system from which to start. Furthermore, the existence of an expository narrative [1] made the work much simpler. One of the functions that this report hopefully will serve is to make it still simpler to produce other systems.

The success of the assembler language bootstrap method can be judged by the fact that there were no delays in the project traceable to it. Again, this success is directly attributable to the existence of a working version of the system in XPL. To have made the translation from another assembly language system would probably have required much more work. Also, the availability of a macro facility made the work much more straightforward, since it reduced the number of decisions that had to be made.

The experience gained in the coding of the locally different routines allows us to comment on the suitability of XPL for the task. At only one point in the entire program is there code that is unwieldy as a result of a missing operator (exclusive-or). Throughout, the readability of the code has been improved by the use of macros for the values of op-codes and other global constants. We feel that this is a most useful technique. We believe that the present XPL is an excellent base for further development of compiler-writing languages. Some suggestions (as yet untried by us) for this appear in Appendix 3.

Finally, we feel that the difference in the sizes of the S/360 and Sigma 5 systems (about 4600 instructions) suggests that a comparative study of the two system architectures would be worthwhile. There is nothing to be gained in terms of improvements to either, but with the coming of easily microprogrammable CPUs, some answer should be obtained as to how much of the difference is due to architecture.

72-12-06

22-12-06

APPENDIX 1

NARRATIVE DESCRIPTION

This Appendix provides a narrative description of the compiler and the RTS module. It is organized into seven sections, according to the principal function of the routines. Within each section, the routines appear in the same order as in the compiler listing to facilitate the use of the Appendix as an aid to reading the code.

Routines marked with '*' are substantially the same as in the original XCOM. Those marked with '**' are identical. In both cases, the reader should consult [1] for additional information.

The discussion assumes the reader has access to a copy of the listing. The grammar for XPL may be found in [1, Ch.6].

Section index

1. Data Declaration and Allocation ... 15
2. Code Generation ... 18
3. Operating System Interface ... 28
4. Lexical Analysis ... 32
5. Symbol Table ... 34
6. Syntatic Analysis ... 35
7. Miscellaneous ... 36

Alphabetic Index to Procedures ... 39

22-12-06

1. Data Declaration and Allocation

These procedures are called as a result of the recognition of a declaration or when a data location is required by the semantics of the program being compiled.

For example, consider the following statement.

declare B(5) bit(8) initial(5, 4, 3);

The recognition of bit(8) as <type> sets TYPE to BYTETYPE which determines the unit of storage allocation. B(5) <type> is then recognized as a <type declaration> and a call to TDECLARE is made, with the value of the dimension as parameter. This value is passed to ALLOCATE, along with a pointer to the <type>. ALLOCATE causes ((5+1)+3)/4=2 words to be set aside. Actual allocation of storage does not occur just yet; two words is the maximum amount that will be allocated. The <initial list>, initial(5, 4, 3) causes successive calls to SETINIT, as each <constant> is recognized. Each call causes EMITDATA to be invoked, where the appropriate object language to effect the actual initialization is generated. Finally, with the recognition of <declaration element>, the declaration has concluded and RESERVE_STORAGE is called to complete the allocation, by initializing any remaining locations to zero.

Semantic requirements are handled more directly, usually with a call to EMITCONSTANT. One special case is that of RESERVE_DATA_LOC, which is used by a number of routines that require the reservation of exactly one location in the data section.

EMITDESC handles descriptors for initialized strings.

EMITSTRING handles the emitting of strings that appear either as constants or with initial. The bit that is returned specifies whether or not a descriptor was emitted. This fact is used to detect '', where a load immediate of 0 may be preferred. If the string is not '',then a descriptor is computed and emitted. The bytes of the string are emitted 16 at a time. As the string control section has byte resolution, CHP is advanced by the length of the string that is emitted.

EMITDATA This procedure is used for the initialization of all data of 32 bits or less in length. The control section, CSECT, is relevant only for allocation of a full word of storage. The declarations of I and ITEM are arranged so that I(4) is the first byte of ITEM. As a result of the adjustment of the value of TYPE, I runs to 7 from 4 or 6 or 7, as the TYPE of

72-12-04

ITEM is BYTETYPE, HALFWORD or FIXEDTYPE. When EMITDATA is called with a TYPE = FIXEDTYPE, the value of ITEM (8 or 16 bits) is right justified. The variable DPB (data pointer byte) acts as a modulo three counter and specifies the next byte in the current word of data being emitted. It is used in the initialization of byte and halfword arrays.

RESERVE_DATA_LOC This is a specialized version of RESERVE_STORAGE. It initializes one word of data storage to zero. The DP value for the word reserved is available in ADRDISP. This procedure is used when the semantics require a zero-initialized variable which, as it is not initialized by the user, should not appear when $D is selected

RESERVE_STORAGE This procedure reserves a block of storage in either the descriptor area or the data area. It may also be called for declared labels (FORWARDTYPE). If the data area is selected, the control section pointer is first advanced to a word boundary. If TYPE indicates a data control section variable, then it is necessary to test DPB to see if a part of a fullword remains unused. This will happen, for instance, if a bit(8) array of size 2 (i.e., 3 bytes) is declared. This check guarantees that after processing of any declaration, DP points to the next fullword boundary.

EMITCONSTANT* For each constant, a linear search of the constant table is made to prevent duplication. Note that if more than 64k of main memory is available, CADD should be declared bit(32).

SMALL_CONSTANT This procedure tests its input for range within the limits for an immediate addressing instruction, and returns accordingly. Use of this procedure allows some small economies, but unfortunately some situations ultimately require the constant to be emitted in any case.

SETINIT Much of the logic of the original version of this procedure has been transferred to other routines. ITYPE is a global variable, set to the current storage type in SYNTHESIZE. Observe how, in the case of character variables, it is necessary to emit the descriptor if EMITSTRING signifies that it has failed to do so. The setting of VAR(MPP1) to HEX(FIXV(MPP1)) allows the initialization of character variables with bit strings. It is thought that this is an easier way to introduce non-alphanumeric strings than setting VAR to the decimal equivalent of FIXV. This alteration was made in conjunction with a (locally) extended version of XPL that supports telytype I/O. Control codes were then declared using this facility and included in output by means of the concatenation operator.

72-12-06

ALLOCATE This procedure has been simplified somewhat from the original, by means of a rearrangement of the values of the type codes. The effect of ALLOCATE is to set NEW(DSP/DP) to DSP/DP plus the number of fullwords required to satisfy the declaration. If these are not all initialized, the call to RESERVE_STORAGE that follows completion of the declaration will fill out the remainder.

TDECLARE* begins the allocation process for type declarations.

SYNTHESIZE (DECLARATIONS) The above procedures are called principally in the course of processing the semantics of declarations; that is, productions that stem from the rule: <declaration statement>::= declare <declaration element>. If the <declaration element> is a <type declaration>, then the difference between NEW(DSP/DP) and DSP/DP reflects the amount of storage reserved by the declaration but not allocated by initial. If the <declaration element> is a macro declaration (literally), then all that is required is to insert the definition, stored in VAR(SP) and name, stored in VAR(MP) into the appropriate tables. Note that the MACRO_NAME table is sorted by length.

The remaining productions associated with data declarations assemble the various items associated with data. Of particular interest is the processing of <type>::=<bit head><number> and the productions stemming from <initial list>. In the first case, it may be seem that the range bit(1) to bit(32) is mapped into three storage types, corresponding to the available hardware addressing types. It might be worthwhile to consider a broader spectrum of types, implemented by appropriate masking operations. True Boolean operations, for instance, are not available with bit(1), for ¬0=255! In the latter case, the type of initialization is passed to SETINIT in the global variable ITYPE. Thereafter each recognition of an initialization item causes a call to SETINIT.

72-12-06

2. Code Generation

This section describes the operation of the routines that are responsible for the generation of the machine code that corresponds to an XPL program. With each recognition of a production in the XPL grammar, SYNTHESIZE is called with the production number as parameter. This results in the generation of machine code, according to the semantics of the language.

As an example, consider the processing of the statement A(1)=A(1)+1; where A has been declared by the statement, declare A(1) fixed;.

A is recognized as an <identifier> by the scanner. The first production to be applied is <subscript head>::=<identifier>(. A is looked up in the symbol table by ID_LOOKUP. When it is found, having been entered by the declaration, the TYPE, FIXL, FIXV, REG and INX stacks are initialized.

The recognition of '1' as a <constant> is made by the scanner. Recognition of it as an <expression> requires the application of several productions, but no action is taken. The next input item is then processed, and we are able to apply <variable>::=<subscript head><expression>

At this point, FIXV(MP)=0, so it is known that we have a subscripted variable, rather than a builtin function. The call to FORCEACCUMULATOR generates a load of 1 into an index register, register 1. Note that nothing further is done with A at this point.

The '=' is now scanned, and the production <replace> ::= =, applied.

The second occurrence of A(1) is processed in a similar manner. We now have a 1 in register 2 as well. Following this, the '1' is recognized as a <term>, and we are able to apply <arithmetic expression>::=<arithmetic expression> + <term>.

A call to ARITHEMIT at this point causes the following action. An indexed load of A(1) into register 2 is generated by a call to FORCEACCUMULATOR. Because the right-hand operand, 1, has TYPE of CONSTANT, an add immediate instruction is used. The value of A(1)+1 is now in register 2.

Without further scanning of the input, we are in a position to apply the production <assignment>::=<variable><replace><expression>. This results in a call to GENSTORE, where store of register 2 is generated, indexed on register 1.

Finally, <assignment>; is recognized as a <statement>, and the register assignments are removed with a call to CLEARREGS.

22-12-06

CLEARREGS* At the end of each statement the registers are all released through a call to CLEARREGS. It will be noted that register zero is not included. This is a result of the fact that it is used for subroutine linkage and function return and is never allocated.

FINDX, FINDG, FINDD, FINDREG These routines allocate a register. If one (TARGET_REGISTER) has been selected, then that is returned. FINDX returns an index register (1 - 7), FINDG a general register (8 - 15) if one is available and an index register if one is not. FINDD returns R for a double register pair, R and R+1. FINDREG does not allocate its result, so that it may only be used in local context, i.e., a load that is to be followed immediately by a store. If no other register is available, zero is returned, so FINDREG always succeeds.

EMITCODE is concerned with placing a completed instruction word in the code control section. STILLCOND ~= 0 indicates that the setting of the condition code that was created by the previous instruction is still available for testing. The emitting of another in instruction cancels this setting, so EMITCODE must set STILLCOND = 0 to recode this fact.

FINDADDRESS sets ADRDISP and ADRSECT for a variable at F in the analysis stacks. If TYPE(F) is not VARIABLE or CNSTYPE we assume a constant. Note that if FINDADDRESS is called for the purpose of generating the storage address of a small constant, it will fail.

EMITIA, EMITMA By packing the instruction components into a word, these routines drive EMITCODE. They are called when an instruction of the immediate addressing or memory addressing type, respectively, is required. Two routines are required because, in general, it is not possible to distinguish between the two types of instruction on the basis of index = 0.

MOVESTACKS* This routine moves the information at position F in the analysis stacks to T. A call to MOVESTACKS precedes the moving of stack top from F to T. An example of the use of MOVESTACKS would be when the information associated with the variable at SP is to be associated with the variable at MP, prior to a reduction. For example, the application of <group head>::= do<case selector>; the <case selector> information is moved to the do location, where it will be used in subsequent processing.

BRANCH If the LOCATION parameter is negative, the branch is relative to a previously declared forward reference. This information is moved to the

control section field, where it will be used by the loader for relocation. COND selects the type of branch and the condition code to be tested for. If COND < 16, then a branch on conditions reset is used, otherwise a branch on conditions set is used. Bits 28 - 31 of COND contain the actual condition code setting to be tested.

CONDTOREG The code that translates a condition code setting into a 0 or 1 in a register is generated here. First, the target register is cleared to 0 by a logical shift right of 32 bits, which does not affect the condition code setting. If the condition, CC, evaluates to true, the branch test 'succeeds' and the next instruction, load 1, is executed. Otherwise, the 0 is left. The condition is saved in STILLCOND in case the branch alone proves to be sufficient, as it will, for instance, when the relational expression being evaluated contains only one operation and its result is being used in a conditional statement, such as if or while. See BOOLBRANCH.

SAVE_REGISTERS, UNSAVE_REGISTERS* These routines are responsible for the saving and restoring of any allocated registers across a procedure call.

CALLSUB, CALLSUB_EXTERNAL These routines generate the appropriate branch instruction for internal and external procedures respectively, as well as doing the saving and unsaving of allocated registers. The difference between the two is the form in which the loader has the entry point information. In the former it is an address in the code control section, in the latter it is an external reference declaration, to be satisfied from some other relocatable object module, which will be made available at load time.

FORCEADDRESS generates a load of the address of the variable at SP in the analysis stack into register R. If this should happen to be a small constant, FINDADDRESS will leave the value of the constant in ADRDISP, so it must be forced into the data area with a call to EMITCONSTANT. If the variable is indexed, the value of the index is added to the address. Note that a word address is computed. Thus if the <variable> is an indexed byte or halfword array, an error will result.

TYPE_OP computes a modified operation code, based on the type of the operand. The addition of "40" or "20" corresponds to the addressing type bit for byte or halfword operands. If TYPE=FIXEDTYPE or TYPE=DESCRIPT, the full-word addressing op code is returned unmodified. A caution is required here. Some operations (e.g., OR, AND) do not have byte or halfword equivalents.

72-12-04

If these are used as actual parameters a different (or illegal) operation altogether will result.

FILE_PSEUDO_ARRAY This routine handles calls to FILE either on the left or the right of an assignment. DIRECTION is a 0/1 code for input/output. FILEP is an index into the analysis stacks for the variable. In particular INX(FILEP) contains the register into which the key (record number) has been loaded and REG(FILEP) contains the register into which the unit number has been loaded. VARP is the address of the source (for output) of object (for input) descriptor of the information to be transferred. The mode parameter is generated from the storage type of VARP. This allows the run-time support routine FILE to distinguish between binary and character records. Note that this distinction depends on the context of the file statement and not on the contents of the file, so that any file may be read in either mode.

PROC_START generates the code for the start of a procedure. This is a branch around the procedure body, as execution of an XPL program begins with the first statement encountered by the compiler, and a store for the return address.

STUFF_PARAMETER generates a store for an actual parameter in REG(MPPi), according to the type of the formal parameter. Note that no type check is made, so that, for example, a character variable may be passed to a parameter declared fixed thus giving access to its descriptor. The check for too many actual parameters is based on the fact that parameters exist in the symbol table with nil names. Note that this test may fail. If procedure A is followed immediately by procedure B with one parameter, then if A has n parameters, the attempt to call it with n+2 parameters will result in the $n+2^{th}$ parameter being 'passed' to B.

FORCEACCUMULATOR When a data item (at P in the analysis stacks) is required in a register (all registers qualify as accumulators in this implementation), FORCEACCUMULATOR is called. The principal alternatives for the data item at P are: VARIABLE, CONSTANT (numeric), CHRTYPE (string constant), ACCUMULATOR and DESCRIPT.

If TYPE(P) is VARIABLE, the alternatives are: a procedure or function call, a SPECIAL item (one of the builtin functions) or a data item.

If we have a procedure or function call, this is indicated by the declared type of the variable (found in STTYPE. FIXL contains a pointer into

the symbol table for P) is CHAR_PROC_TYPE (for procedures declared to be of type character) or LABELTYPE (for all others). The number of actual parameters has been accumulated in CNT. If the variable in the symbol table at CNT(P)+1 is nil, then not all parameters have been passed. (This test is not foolproof. See comment to this effect under STUFF_PARAMETER.). Upon return from a function procedure, it is necessary to allocate a register for the result of a function, and load the result from register zero, where it was placed by the return statement. In the case that FORCEACCUMULATOR was invoked from call statement processing, this load is superfluous and will be rescinded.

Within the SPECIAL alternative, there is an alternative for each builtin function that may appear on the right of an assignment operator but is not handled by SYNTHESIZE. The particular function is indicated by the value of SYDISP, which is initialized to a unique value for each function. The functions INPUT, FILE and COMPACTIFY are handled with procedure calls to RTS. Note that for INPUT, a parameter value of 0 is generated if none is already available. The EXIT function is handled by the generation of an abort CAL in line.

Finally, we come to variables that are data. If the variable is indexed (INX(P)<>0), then the register that has already been loaded with the index value is used, otherwise a general register is selected. Note that in all cases, the TYPE is set to ACCUMULATOR or DESCRIPT, so that a subsequent call to FORCEACCUMULATOR for this variable has no effect, except possibly to move it to another register. Additional processing for new builtin functions should be added, with appropriate logic to test SD, just above the call to ERROR.

The CONSTANT and CHRTYPE alternatives are treated in a straightforward manner. The constant item is emitted as data and its value or descriptor loaded into a register. If the CONSTANT is a small number (between "80000" and "7FFFF") it is loaded without first emitting as data with a load immediate instruction. If the string is nil, EMITSTRING returns false, and a nil descriptor is loaded, otherwise the descriptor is loaded from the descriptor area, where it was placed by EMITSTRING.

A TYPE of CONDITION occurs when the production <logical primary>::=<string expression><relation><string expression> is recognized. If a value for this is required, as opposed to the condition code setting it generates, then CONDTOREG is used to generate instructions to put a 0 or 1 in the register.

72-12-06

Finally, TYPE of ACCUMULATOR or DESCRIPT means that the item in question is already in a register, and the only possibility is that it is not the correct one.

FORCEDESCRIPT is called when a descriptor is required in an accumulator. If TYPE of DESCRIPT does not result from FORCEACCUMULATOR, a number-to-string conversion is performed. Thus, for instance, BYTE(1) will generate a load of binary 1, followed by a conversion to the character string '1', followed by a load of the 0th byte of this string, or "F1".

GENSTORE handles the generation of a store operation for an item in a register. If the TYPE of the item on the right of the assignment is SPECIAL, however, it is assumed that the store operation has already been carried out. An example of this is the INPUT operation, where the string move operation has already been carried out at this point. Otherwise, there are two principal alternatives: either the item on the left of the assignment operator is a builtin pseudo-variable, and is of TYPE SPECIAL, or it is a data item.

If it is SPECIAL, the particular pseudo-variable is indicated by the value of STDISP, as described for FORCEACCUMULATOR. The possibilities are BYTE, OUTPUT or FILE. For BYTE, we must generate a byte store operation, based on the string control section. First, the byte to be stored is brought into a register. If BYTE has two operands, the second (offset) is in the register INX. REG contains the descriptor. The sum of these two, or the descriptor itself if there is no second argument, provides the index for the store. Note that the address generation hardware will ignore the length bits in the descriptor so there is no need to mask. If the pseudo-variable is OUTPUT, then it is necessary to load the default value zero for unit, if none has been selected. FILE is handled by the FILE_PSEUDO_ARRAY procedure. Note that the statement FILE(,) = FILE(,); will not be handled correctly because the type of the righthand FILE is SPECIAL and will cause the lefthand one to be errored.

For a VARIABLE, the only consideration is the possibility of a character variable, which requires a store in the descriptor control section. Again, note how automatic conversion from numeric to string takes place, if necessary, for character assignments. The FORCEACCUMULATOR call generates a load of the right-hand side of the assignment, if it is not already in a register.

ARITHEMIT This routine emits the instructions for most arithmetic expressions. After forcing the lefthand operand into a register, the following

alternatives are considered.

If TYPE(MP) = DESCRIPT, then the left operand is a character variable. In this case, the only permissible operation is comparison, for which a call to the COMPARE routine in the RTS module is generated. Note that this includes bit variables of length greater than 32.

If the left operand is FIXEDTYPE (i.e., a fullword) and a VARIABLE, then the operation can be performed directly from storage. This is not optimal, for some halfword operations from storage are possible. In any case, it may be observed that bit(1) - bit(16) variables are not economical, for although data storage is saved, arithmetic may need an extra load operation, as compared with fullword variables.

If TYPE(MP) = CONSTANT, then if an immediate addressing operation is possible it is used, otherwise the constant is placed in the data area and a normal operation used. Finally, the right operand must be loaded by FORCEACCUMULATOR.

BOOLBRANCH generates code for while and if conditional branches.

The first alternative, STILLCOND = 0, implies that the condition code is still available for branching. To use it, however, we must first cancel the effect of CONDTOREG by backing up the object code pointer, over the load of 1/0. The value of STILLCOND is the value of the branch code, set when the relational expression was recognised.

Another opportunity for peephole optimization [1, p.223] is available on the second alternative. If the relational expression is a constant, then either it is true or false. If true, then no branch will be required, recorded by FIXL(MP) = 0. See FORWARD_LOCATION for use. If false, a branch unconditional is used.

Finally, if the type of the expression is not CONDITION, code is generated to test the least significant bit of the expression. The forward reference number that is saved in FIXL(MP) will be passed to FORWARD_LOCATION when the destination of the branch is encountered.

SET_LIMIT generates code for the iteration control of a do statement. In the case that the control is not a constant, note that the call to FORCE-ACCUMULATOR, at this point, means that the control is evaluated once only, at first entry to the loop.

SHIFT_CODE handles code generation for SHL and SHR. For right shift the negative shift count is used. If the number of bits to be shifted (in FIXV(MPP1)) is a constant, the shift count is included in the instruction. If not, then it must be loaded with a call to FORCEACCUMULATOR, and used as an index. Note that only bits 24-31 of the index are significant.

SYNTHESIZE This section describes those cases of SYNTHESIZE that do not deal with declarations. It is organized according to statement type and attempts to give an overview rather than a comprehensive description.

if...then...else The principal feature of the semantics for these productions is the assignment of forward reference numbers to the then and else locations and their subsequent satisfaction through calls to FORWARD_LOCATION.

do...end The do...end pair make up a group. Semantic processing takes place at the head, where there is a different production for each type of do and at the end. The four types are empty (;), step, while and case and they are identified by the assignment of 0, 1, 2 and 3 to INX, respectively.

step involves the <step definition> and the <iteration control>. The <step definition> causes the assignment of some <expression> to the loop control variable, followed by the code required to increment and test it. The <expression>is brought into a register by FORCEACCUMULATOR. We then branch around the next two instructions, which generate the load and increment for the controlled variable that are executed on subsequent passes through the loop. PPSAVE is set to the location of the first of these instructions. The final store instruction puts the current (initial or incremented) value of the controlled variable in memory. The compare and branch greater provide the termination of the loop. Observe that the controlled variable is greater at termination by one increment. The increment and termination values are set by the semantic processing for <iteration control>. These determine whether the default increment value of 1 or the value of an <expression> is to be used. Note again (refer to SET_LIMIT for details) that the values on initial, increment and final values for the iteration control are evaluated only at the beginning of the loop and that they are evaluated in reverse order, so that if they involve procedure calls, side effects must be watched carefully.

while processing is much simpler, as it involves only the generation of a test and branch sequence for the value of <expression>. Note that BOOLBRANCH generates no code if the <expression> is constant and the least significant bit is set.

22-12-06

case processing involves the evaluation of an <expression> and a branch indirect on a table of addresses, whose first location is the forward reference that is declared. An indexed load followed by a branch is used because of the post-indexing [2, p.10] feature of the Sigma series. The second forward reference is that of the end of the case statement. Each case terminates with a branch to this point.

procedure processing may be separated into three parts: <procedure head>, <parameter list>, and <procedure definition>. <procedure head> processing saves information about the current (now previous) procedure in its own symbol table locations. PROC_START is called to generate the branch around the procedure and the store of the return address. <parameter list> processing creates symbol table entries for the parameters that contain the name only. Other information is filled in at the time of declaration. Note that this means that the entries for the parameters always follow the procedure name in the symbol table, but that the order in which they are allocated storage depends on the order of declaration. When the syntatic entity <procedure definition> is recognized, the entire procedure has been processed. At this time the symbol table local to the procedure (from PROCMARK to XDECST) is checked for labels that have been declared but have not appeared. Remember that the label type declaration requires that the label appear within its scope. Finally, the parameters themselves are checked. If SYTYPE = 0 then no declaration has appeared. Once this is done, the name of the parameter is discarded by setting SYT(I) to nil. Thus, these names will not conflict with other declarations, but the declaration information is available for procedure calls. Finally, the current procedure is reset from information stored in the symbol table, the return jump emitted and the forward reference of the jump around that began the procedure is satisfied.

return requires only an indirect branch to the location at which the return address was saved. Optionally, FORCEACCUMULATOR is called to load the returned <expression> into register 0.

call statement processing occurs principally in FORCEACCUMULATOR. However it is necessary to rescind the code emitted to load the result of a function into some register other than 0. As the <variable> may also be a builtin function or undefined, the specific test is necessary.

go to statements involve only a simple branch, unless the target label is undeclared, in which case it is entered into the symbol table with a forward reference number.

72-12-06

<expression> processing, with the exception of that associated with <variable> takes place primarily in FORCEACCUMULATOR and ARITHEMIT. The only production that might need clarification is that of <logical secondary>::= <logical primary>. If TYPE(MP) = CONDITION, then the bit that indicates BCS/BCR in the branch code is complimented. A logical exclusive or operator would be useful here.

<variable> processing includes that associated with some builtin functions, procedure calls and subscripts, as well as the initial lookup of the identifier. Argument list processing takes place in the productions <subscript head>::= <subscript head><expression> and <variable>::=<subscript head><expression>). In both, the <identifier> that begins the <subscript head> may be a declared variable or builtin function, the distinction being made on the value of FIXV. In the former case, processing is straightforward.

In the latter case, the particular builtin function is detected by the value FIXV, which is set from SYDISP in ID_LOOKUP. Builtin functions with SYDISP >= 8 that have one or two arguments and are not otherwise handled have their arguments loaded into registers that are pointed to by REG and INX. The only builtin function that is handled in this manner is FILE, but the user may easily insert his own using this mechanism. Note, however, that if there are more than two arguments, the call will be rejected. Particular features of other builtin functions are noted below.

length The length field is loaded from the first byte of the descriptor without modification, as true length is used.

substr A descriptor for the first argument is forced. If the register is zero, then the descriptor resulted from a conversion and must be saved.

byte If there is only one argument, and that is a character string constant, then a load of the character in question is used. Otherwise, a byte load is used, indexed if there are two arguments.

3. Operating System Interface

These procedures provide the interface with the (RBM) operating system. The interface is at two levels: object language and run-time.

The object language procedures are responsible for generating the Sigma standard object language that corresponds to the compiled XPL program. The interrelationships among these routines may be seen in Figure 8. Their effect may be observed by selecting the $B listing control.

The run-time procedures are coded in MACROSYMBOL and are linked to each XPL program at load time. They provide an easily modified interface between XPL and the supporting operating system. These procedures are described in Appendix 2.

OBJECT LANGUAGE

GETLOADCODE The object (load) code record is composed of three parts: 4 bytes of header information, 108 bytes of object code and 12 bytes of sequencing. The latter is zero for every record. The object code portion of the record is constructed byte by byte by the LOAD_CODE_BYTE procedure. When the current record is exceeded, GETLOADCODE is invoked. This sets the sequence number (record number mod 256) and computes the checksum. The reason for printing both record number and sequence number is that for a large program, the latter is not unique. This feature simplifies the debugging of load code. The format of the header is described in detail in [4, Appendix A].

LOAD_CODE_BYTE The principal purpose of this routine is to manage the placement of successive bytes of object (load) code in the LOAD_CODE array, calling GETLOADCODE as necessary when the array bounds are violated.

In addition, a partially interpreted trace of the object code emitted is provided under the $B option. It is desirable to have the load code byte(s) followed by the information whose loading they govern. This is accomplished with the variable CURRENT, whose value determines the context in which the next byte is to be interpreted. A value of zero causes interpretation as an object code operator. For each such, the number of following bytes (FB) to be printed is determined. FB counts from zero; FB=0 means that 1 byte follows. CURRENT=1 causes the current byte to be added to the line being constructed. CURRENT=2 is used by load code operations that specify the number of bytes to be loaded in the byte following the operation. CURRENT=3 is used by

operations where there is variable length information following the operation, in addition to the bytes to be loaded. Full discussion of the various object code operators may be found in [4, Appendix A].

LOAD_CODE_WORD transmits four bytes of information to LOAD_CODE_BYTE. The declarations of I and W are placed so that I(4) is the first byte of W, thus allowing W to be accessed as bytes.

LOAD_RELOCATABLE computes the appropriate object code for relocatable loading. CSECT < 0 indicates a forward reference, for which bit 5 in the load relocatable operation is set. If the control section exceeds 255, 2 bytes of information are required for name specification, and bit 4 of the operation must be set. The object code and control section are then emitted. "Control section" in this context includes all declarations for which there is an associated declaration number.

DEFINE_RELOCATABLE_ADDRESS emits object code for an address expression. An address expression allows the loader to compute an address for purposes of relocation. The add value of declaration (AVD) code determines the base address to which is added the constant value (AC) of the designated control section pointer (CSP). The expression is completed by an end code (XEND).

USECT This procedure performs a function that is equivalent to the MACRO-SYMBOL directive of the same name. If the current control section (established by some previous call) is not the one required, a new one is selected by an origin (ORG) code followed by an address expression. This causes the loader to update its pointer for the designated control section.

REFERENCE This procedure generates loader declaration for global and external declarations. It also keeps the external reference counter. This value is returned to be used in referring to the declaration for addressing purposes. For the routines in the RTS module, the external reference is kept in a variable whose name is the routine name followed by 'NTRY', e.g., INPUTNTRY.

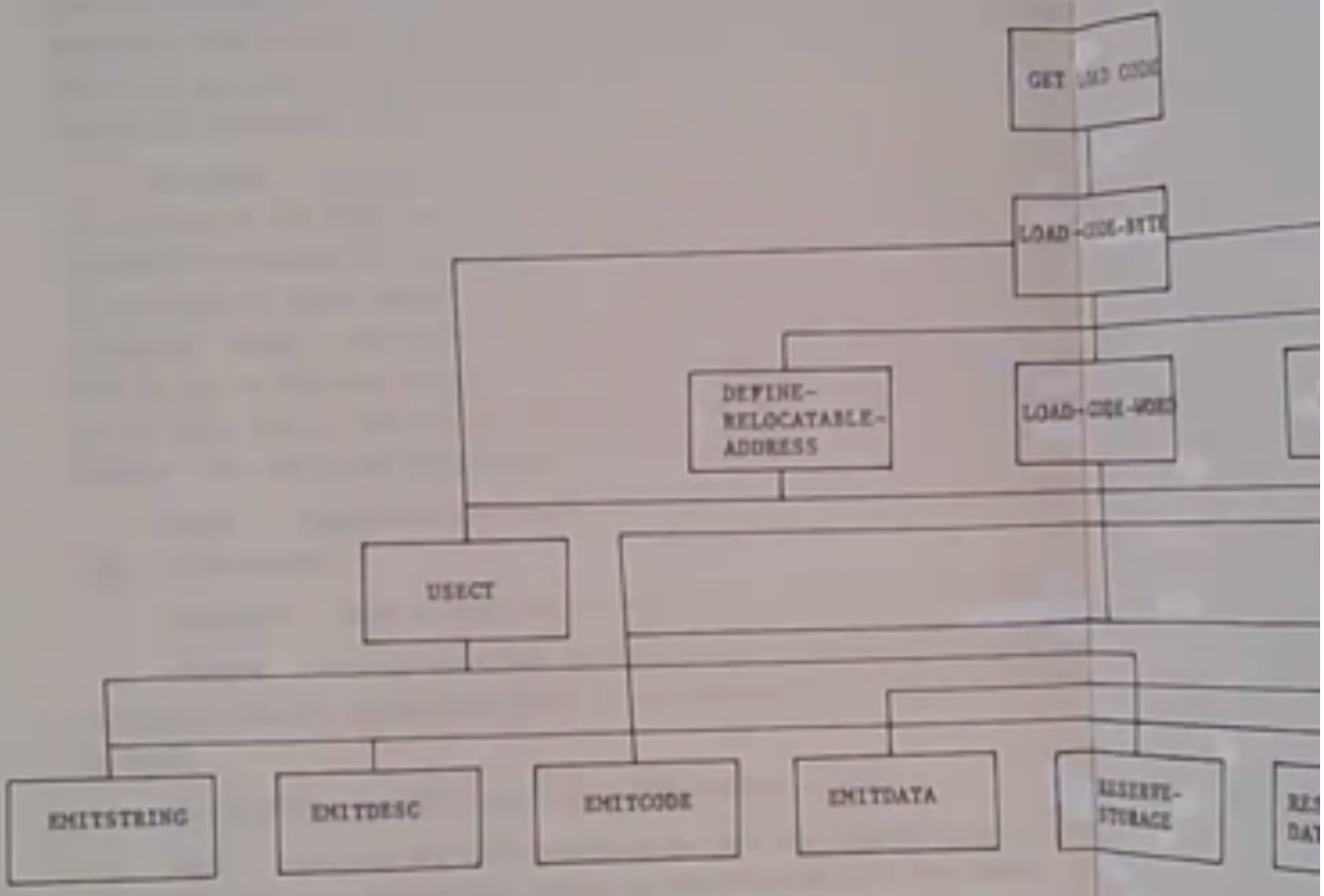
FORWARD_LOCATION One of the principal economies realized by using the loader is that no 'fixups' of forward references are required. Instead, each forward reference is assigned a (negative) number. When the forward reference declaration is encountered by the loader, all references to it are updated. The test FORWARD_REFERENCE=0 detects situations where no branch was

22-12-06

used. See BOOLBRANCH. The temporary swap of CSP(CSECT) and VALUE allows the definition of non-current forward values, such as in <if statement> processing.

GLOBAL_LOCATION provides the definition facility for global names. The address provided by the definition code (DEFD) is made available by the loader to other modules being loaded. This facility is used to pass the addresses of the string pointers (FREEPOINT, etc.) to the RTS module.

22-12-06



OBJECT CODE PROCEDURES

Figure 1

BJECT CODE PROCEDURES

Figure 8

4. Lexical Analysis

The function of the lexographic analyzer is to break the text of the program being compiled into characters and to then reassemble them as syntatic tokens. There exists a formal syntax that underlies this process and it would be possible to have a table-driven scanner which would operate in a way exactly analogous to the parsing routines. The reason that this is not generally done is that such a syntax involves a large number of productions which are not overly complex. As a result it is usually more economical to encode the decisions in the logic of a procedure.

GET_CARD* This procedure is considerably smaller than the original as a result of the fact that it is no longer necessary to pre-process the COMPACTIFY procedure for each program compiled. The test LBAL>0 is made to determine if there remains any macro expansion that has not yet been processed, where the remaining expansion is returned in TEXT. Otherwise, TEXT is set to the next source card. In the generation of the listing line for $M (Max. list), the use of the CSECT_BASE array to allow the absolute address for the code generated to be printed on the listing line.

CHAR** increments the pointer into TEXT, CP, and calls GET_CARD if TEXT is exhausted.

DEBLANK** used to scan over blanks within bit ("...") strings.

BCHAR** used to fetch the next bit string character into CHR. Processing of the bit string base width is also handled, to make it transparent to SCAN.

BUILD_BCD** used to build long bit strings in BCD.

SCAN uses the GET_CARD procedure to read the program text as a stream, i.e., ignoring card boundaries, except as specified by $|. Its result is returned in: BCD, the symbol that was found, NUMBER_VALUE, its binary equivalent if a numeric or short bit string constant, and TOKEN, the syntatic token that represents the symbol

The "outer loop" of SCAN is a case statement which has one case of each class of lexographic entities. These are: illegal characters, blanks, character strings, bit strings, identifiers (including reserved words and macros), numbers, divide-characters (/ and /*), special characters and or-characters (| and ||). The array CHARTYPE determines the class for each of the 256 EBCDIC characters. It is initialized in the procedure INITIALIZE.

Within these alternatives, the procedure logic deals with the special requirements of each, such as ensuring that strings do not exceed 255 characters. Of particular interest is the identifier section. This logic is critical, as every identifier passes through it. Once the identifier has been assembled in BCD, it is checked to determine if it is a reserved word or macro identifier. If the former, the unique syntatic token is returned. If the latter, the macro is expanded and the scan restarted at the beginning of the expanded text.

5. Symbol Table

These routines are responsible for the management of the group of tables that encode the information concerning all declared identifiers in the program being processed. The speed of the algorithm that is used to search the symbol table for a particular identifier is one of the determining factors of the speed of the compiler, as it is used every time the syntatic class <identifier> is found in the program, except for reserved words and macro identifiers.

HASHER** This procedure has been added by the original authors after publication of their book [1]. It removes one of the major drawbacks of the original XCOM by the use of the hash-addressing to speed up the symbol table search.

ENTER* searches the symbol table above PROCMARK for the identifier name, using the hash links kept in the PTR stack. Note that even at the outermost level of the program, i.e., outside all procedures, the search does not extend into the initialized part of the symbol table. Thus these identifiers may be redeclared, or, looking at it from the other side, inadvertently lost by declaration of like-named identifiers in the user's program. For labels, ENTER may be called twice, once at the declaration of the label or its occurrence in a go to and a second time when the label itself appears. When this happens, FORWARD_LOCATION is called to satisfy forward references. If any of the symbol table information is not available at the time ENTER is called, i.e., when the identifier first appears, it is entered when it becomes available, usually by ALLOCATE.

ID_LOOKUP* searches the symbol table, using the hash chain in PTR. Note that the hash chain is followed to the end, irrespective of scope, but that the latest occurrence of the identifier will be found, because of the stack property of the symbol table. Undeclared identifiers are flagged by setting FIXL(P) = -1.

UNDECLARED_ID* processes undeclared identifiers by emitting an error message and making a default entry in the symbol table as a fixed variable.

6. Syntatic Analysis

These routines contain the logic that determines if the program presented to the compiler is compatible with the XPL syntax. They are described in considerable detail in [1].

RIGHT_CONFLICT** checks for legal right context.

RECOVER** attempts to pick up compilation after an error has been discovered.

STACKING** decides if a token is to be stacked or if a reduction is required first.

PR_OK** Context check for equal or embedded right parts.

REDUCE** decides if the proposed reduction is allowable, and calls SYNTHESIZE if it is. Note that it is here that MP, MPP1 and SP are set.

12-12-06

7. Miscellaneous

These routines perform service functions or control the overall flow of the compiler.

PAD** pads its parameter on the right with blanks, providing that the total field width requested is not greater than the length of the parameter string. It is used for producing left-justified alphanumeric output. The advantage of the pad strategy, over the conventional 'A' format is that meaning is preserved at the expense of neatness, when unexpectedly long output is encountered.

I_FORMAT** converts binary input to decimal string, right-justified. The action taken for input that is excessively long is the same as with PAD, and the same comments apply.

X_FORMAT the analog of I_FORMAT for hexidecimal output. If a field width of more than 8 digits is requested, an error results.

XB_FORMAT X_FORMAT with leading blanks instead of leading zeros.

ERROR causes the current line of source to be printed, if listing has not been requested. This line is followed by a pointer line. The position of the pointer indicates the last-scanned character and, therefore, the general location of the error. The value of the severity parameter may be questioned, on the grounds that any error constitutes a deviation from the intent of the programmer, even if a 'reasonable' recovery may have been made.

C_LINE This procedure constructs the MACROSYMBOL format line for instructions. The first test determines if the op code specifies indirect addressing, and sets the op code portion of the result accordingly. The remainder of the op code is computed with the aid of OP_NAMES and the OPER arrays. OPER(I) gives the byte in OP_NAMES where the name of the Ith op code begins, thus allowing the name to be isolated. The program address computation permits the absolute address of the code emitted to be printed. This is possible because the base of the code control section, that is the absolute address at which loading commences, is known.

INITIALIZATION* The initialization that is performed in this routine precedes any actual compilation. It is essentially constant for one language and might better be incorporated in the declarations themselves. One way of accomplishing this would be to extend the ANALYZER program to incorporate the appropriate information and to produce initialized arrays. The effect of such

a modification would be to integrate the table building function more closely and to better identify those functions that are constant over a language.

The same criticism applies to the generation of the object language prelude by the calls to REFERENCE. A better approach would be for the compiler to read the appropriate records from a permanent file and to transfer them directly to the LCFILE.

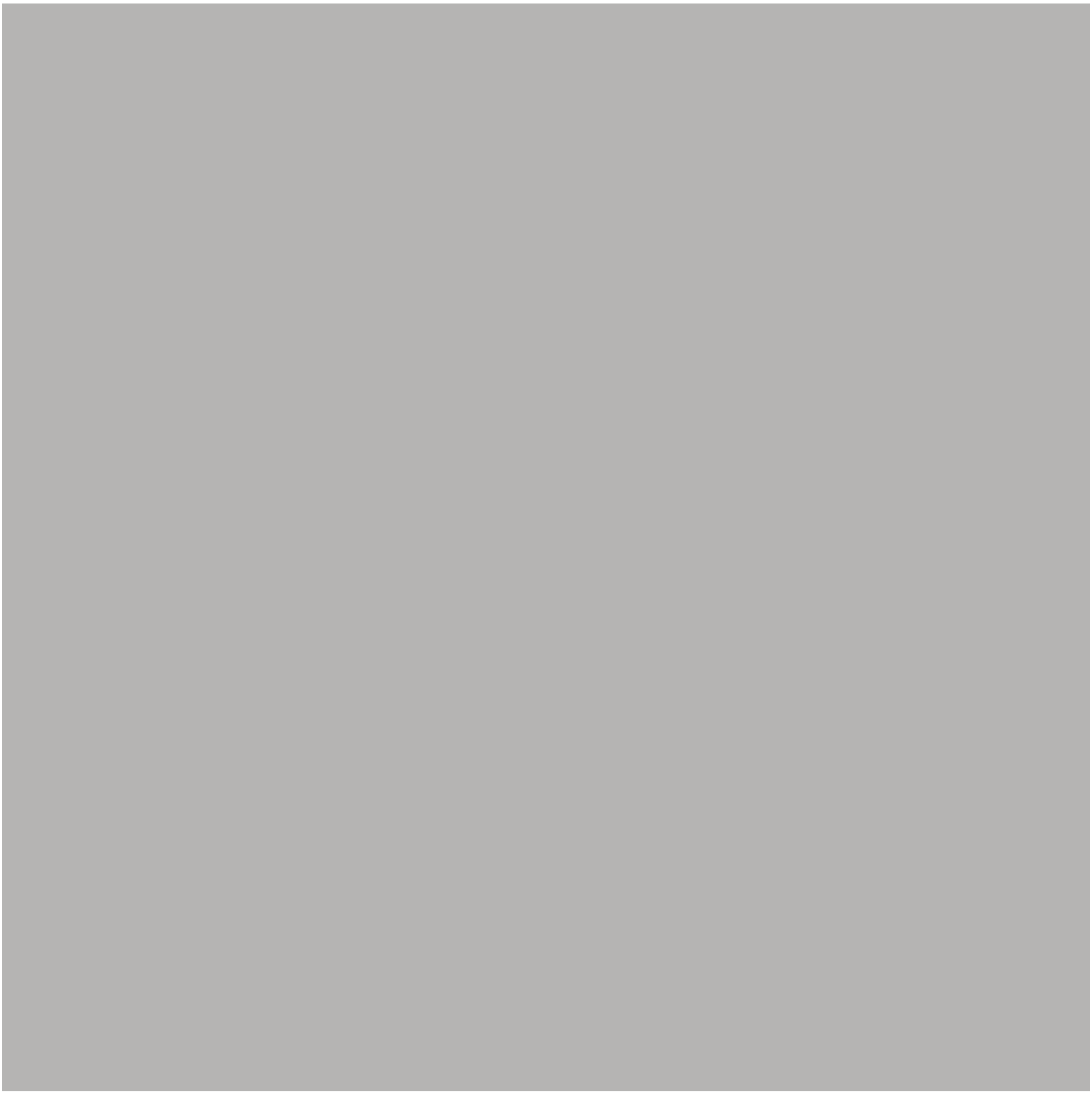
STACK_DUMP** With this routine, the user is informed exactly where in the syntax the compilation process broke down. It is valuable both in debugging new grammars and in providing accurate information about failures.

COMPILATION_LOOP* The main body of this procedure governs successive steps in the compilation process. The while block that begins "do while COMPILING;" continues until COMPILING is set FALSE either by the recognition of the syntactic construct <program> or by the detection of 25 severe errors. This block governs the interaction between SCAN (to discover syntatic tokens), STACKING (to associate them into productions), REDUCE (to apply the production), and SYNTHESIZE (to apply the semantics associated with the production). Note that for each token, the analysis stacks are initialized here. One should note also the assignments at the end of the "do while STACKING;" loop. They constitute the basic information concerning a syntatic construct that is carried into SYNTHESIZE.

The section that follows the compiling loop 1) emits the code that returns the control to the monitor, 2) finishes off the object language with the definition of the value of NDESCRIPT and an end-of-module code. Finally, there are the nonstandard control section declarations, which inform the loader how much core is required for each.

SYMBOLDUMP* produces a dump of the symbol table at the level which it is called. That is, if called in a procedure, only those variables local to the procedure are dumped. If called from the main program, all user-defined globals are dumped. If $S is in force at program termination, all program globals are dumped. In addition, all values for the relocation of program sections are known, so all addresses are absolute. The format of the dump has been modified in order to reduce the amount of repetition. One notable feature of this procedure is the STRING_GT routine, which does an 'honest' string compare. That is, compares two strings of unequal length on the basis of minimum length. This is the more common notion of string comparison.

72-12-06



DUMPIT* This procedure produces the value of certain program counts and instruction use frequencies. It has been retained on the grounds that it constitutes an important part of the original XCOM, but its utility on a continuing basis may be questioned. Once the structure of the compiler has been finalized, it does not appear to be of interest to compute call frequencies.

MAIN PROGRAM As INITIALIZATION and COMPILATION_LOOP are called just once, the division of responsibility is somewhat arbitrary. If $S is in effect thru the end of the program, the values for CSECT_BASE are computed and passed on to STMBOLDUMP, thus providing the absolute addresses for all items dumped.

8. Alphabetical Index to Procedures

ALLOCATE 17
ARITHEMIT 23
BCHAR 32
BOOLBRANCH 24
BRANCH 19
BUILD_BCD 32
C_LINE 36
CALLSUB 20
CALLSUB_EXTERNAL 20
CHAR 32
CLEARREGS 19
COMPILATION_LOOP 37
CONDTOREG 20
DEBLANK 32
DEFINE_RELOCATABLE_ADDRESS 29
DUMPIT 38
EMITCODE 19
EMITCONSTANT 16
EMITDATA 15
EMITDESC 15
EMITIA 19
EMITMA 19
EMITSTRING 15
ENTER 34
ERROR 36
FILE_PSEUDO_ARRAY 21
FINDADDRESS 19
FINDD 19
FINDG 19
FINDREG 19
FINDX 19
FORCEADDRESS 20
FORCEACCUMULATOR 21
FORCEDESCRIPT 23
FORWARD_LOCATION 29
FORWARD_REFERENCE 29
GENSTORE 23
GET_CARD 32
GETLOADCODE 28
GLOBAL_LOCATION 30
HASHER 34
I_FORMAT 36
ID_LOOKUP 34
INITIALIZATION 36
LOAD_CODE_BYTE 28
LOAD_CODE_WORD 29
LOAD_RELOCATABLE 29
MAIN_PROGRAM 38
MOVESTACKS 19
PAD 36
PR_OK 35
PROC_START 21
RECOVER 35
REDUCE 35
REFERENCE 29
RESERVE_DATA_LOC 16
RESERVE_STORAGE 16
RIGHT_CONFLICT 35
SAVE_REGISTERS 20
SCAN 32
SET_LIMIT 24
SETINIT 16
SHIFT_CODE 25
SMALL_CONSTANT 16
STACK_DUMP 37
STACKING 35
STUFF_PARAMETER 21
SYMBOLDUMP 37
SYNTHESIZE 17, 25
TDECLARE 17
TYPE_OP 20
UNDECLARED_ID 34
UNSAVE_REGISTERS 20
USECT 29
X_FORMAT 36
XB_FORMAT 36

APPENDIX 2

THE RUN-TIME SUPPORT MODULE

The run-time support module incorporates all the functions of the original submonitor [1, Appendix 1] inline support routines that appear in the original XCOM, and COMPACTIFY.

The RTS module is an interface between the particular needs of the XPL program and the more general capabilities of the RBM operating system. It also isolates those features that are dependent upon the operating system.

The RTS routines fall into two classes, I/O support and XPL support. The former, INPUT, OUTPUT, and FILE, translate the XPL program's requests into requests to RBM. In FILE, we simulate random access to blocked files. This was necessary as insufficient disk was available to permit large files of one granule per record. The direct access facility provided by RBM does not permit blocked random files. All records are assumed to be one granule in length. This results in considerable waste of storage when record size is small compared to granule size, which may be as much as 256 words depending on the type of disk in use. Error and abnormal returns are intercepted and processed. The XPL support constitutes functions that are built into each program by the original XCOM.

Steps in the addition of new builtin functions.

1. Introduce the name of the function into the symbol table, SYT. Add type of SPECIAL to SYTYPE and the next integer in sequence to SYDISP. Increase the initialization of PROCMARK and NDECSY by 1. Add a variable nameNTRY, where "name" is the name of the new builtin function.

2. Add the assignment: nameNTRY = REFERENCE('name', REF); to INITIALIZATION.

3. Add the appropriate code to process the references to the new function. Generate calls with: CALL CALLSUB_EXTERNAL(nameNTRY);. The discussion of FORCEACCUMULATOR in Appendix 1 indicates how this is done.

4. Add the appropriate code to the RTS module, with "name" as the entry point of the new function.

Run-Time Support Procedures

PROLOGUE computes the value of the string pointers. All otherwise unallocated core is allotted to the free string area (FSA).

INPUT sets FPT parameters to read into FSA and builds descriptor for resultant input. One feature of INPUT processing that some users might find unsatisfactory is the rule that only one end-of-file condition per unit is permitted. This feature was designed to avoid the aborting of a job by RBM in an attempt to read past an end-of-file. Multiple files can still be processed, if separated by !EOD's. Once NO_EOF(unit) is set to zero by an end-of-file, the next access will produce an XPL abnormal abort, which gives the end-of-file error code.

OUTPUT transfers parameters from descriptor of string to FPT. Note that the actual length of the string is used, in distinction to INPUT where 80 byte records are assumed. For proper communication between OUTPUT and INPUT on the same file, output records should be padded to 80 characters with blanks.

FILE In order to provide for 'random' files of less than one granule record size, it is necessary to use the file positioning facilities of RBM. The file concept has been extended to include character string I/O, as determined by the context of the file statement. The record size is determined by the created size of the file, 'actual byte count' being used for length on string input.

DEC provides conversion from binary to decimal. The result is a string with optional minus sign and no leading blanks.

HEX provides conversion from binary to hexidecimal. Result is significant digits only, with no leading zeros.

COMPARE does a byte by byte comparison of strings of equal length. Note that the result of the comparison is returned in the condition code, so that the generated code is the same as if the comparison were accomplished with a string compare instruction.

CATENATE A check is made to determine if either string is nil, in which case no concatenation is necessary. In addition, if the left hand string in the operation is the last allocated string, only the right hand string need be copied.

CK_CMPCT This procedure is only accessible internally. It serves to localize the logic for the check if FREEPOINT has passed FREELIMIT.

COMPACT This is just an encoding of the original COMPACTIFY procedure, modified to discard zero-length descriptors as well as full-word-zero descriptors.

APPENDIX 3

EXTENSIONS TO XPL

We list here some changes and extensions to the language XPL that have occurred to us in the course of the work of implementation.

Global and External Declarations

These would provide a facility to communicate with other routines without incorporating them into the symbol table of the compiler. It would be advisable not to make this a general facility, but to limit it to routines using the XPL calling sequence.

Items Legal for Initial

At present, only constant items may be placed in initial lists. We would extend this to names (addresses). In addition, we would have a facility for describing the exact field within the storage unit.

Use of a Stack

Use of the "temp" stack provided by the Sigma architecture would improve core utilization and permit recursion.

String Variables

It should be possible to describe a non-dynamic string type. This would be of considerable use in situations, such as listing a deck of cards, where information is discarded at once, leading to unnecessary compactification. One solution to this would be to extend INPUT and OUTPUT to handle binary information. A type transfer function could be used for conversion between byte arrays and character strings.

Separate Compilation of XPL Routines

This problem is complicated by the dynamic management of strings, which requires descriptors to be in contiguous storage. However, a limited separate compilation facility would encourage program modularity and save on recompilation time.

Constant Arithmetic and Addressing

To load a constant for subscripting or to add one constant to another is wasteful. It does not appear to be a difficult task to provide compiler facilities for this.

Simple LR(1) Parsing

The SLR(K) scheme of DeRemer [5] promises improvement over the mixed strategy precedence now in use.

Magnetic Tape Manipulation

Builtin functions that permit access to standard magnetic tape manipulation facilities such as rewind, skip, etc., would be useful.

APPENDIX 4

LANGUAGE DIFFERENCE: XPL/360 and XPL/I5

Construct	/360 Action	/I5 Action
Maximum string length	256	255
character initial (<number>)	<number> to dec.	<number> to hex.
HEX function	N.A.*	Converts argument to hexidecimal string
STRING variable	N.A.	Access strings as byte array
TIME, TIME_OF_GENERATION, DATE, DATE_OF_GENERATION, MONITOR, CLOCK_TRAP, INTERRUPT_TRAP	Available	N.A.
TRACE, UNTRACE	Available	No trace available
FILE with character variable	No	Yes
Maximum identifier length	256	255
INPUT record length	In DD statement	80 characters
OUTPUT record length	In DD statement, padded if short	String length, no padding
Forward reference to procedures	Yes	No
Error message if string truncated by concatenation	Yes	No
FILE, legal units	1 - 3	0 - 3

$-Toggles

Toggle Letter	Meaning
B	Interlist object language.
C	Interlist emitted code, SYMBOL format
D	Interlist initialized data.
L	Standard list, line number and symbolic
M	Augmented list with code address and procedure name.
S	Dump symbol table
T	Dump statistics tables.
|	Set margin at column.

Initially, all toggles, except | are set false. | is set to 80.

---

* N.A. - Not available

APPENDIX 5

INTERPRETATION OF XPL LISTINGS

Notes to the Sample Program

(1) $-toggle letters may appear on the !XPL card.

(2) This column has the address for the item being described. If it is a code line, the address is absolute, otherwise it is relative to the appropriate control section. In this case, the first three lines are relative to the data section, the fourth is relative to the descriptor section and the subsequent lines are code and therefore absolute.

(3) This column has the value of the item, if it is a data or descriptor item.

(4) This column has the value of the control section to be used for relocating the line item. It appears only on data and code lines, so that the descriptor line may be identified by the absence of this item. 0 means an absolute item. 1 through 4 indicate one of the four control sections used by the compiler. A number greater than four indicates an external reference to a run-time support routine. Negative numbers are forward references.

(5) Code lines are printed in MACROSYMBOL format. Again, addresses are relative to the specified control section, except when the control section is 1 (code), when the address is absolute.

(6) These three instructions are used to translate the condition code setting into a 0 or a 1 in a register. In this case, a numeric truth value is not required and the condition code may be used directly for the branch. As a result the compiler 'backs up'. This is indicated by the addresses of the generated code.

(7) This branch is relative to a forward reference. The destination of the branch is the end of the while statement. The forward reference will be defined when the corresponding end is encountered.

(8) Call to the external routine DEC, which performs binary-to-decimal conversion.

(9) This is a descriptor for the string ' '. The first byte contains the length and the last three bytes contain the byte address of the first

APPENDIX 5

INTERPRETATION OF XPL LISTINGS

Notes to the Sample Program

(1) $-toggle letters may appear on the !XPL card.

(2) This column has the address for the item being described. If it is a code line, the address is absolute, otherwise it is relative to the appropriate control section. In this case, the first three lines are relative to the data section, the fourth is relative to the descriptor section and the subsequent lines are code and therefore absolute.

(3) This column has the value of the item, if it is a data or descriptor item.

(4) This column has the value of the control section to be used for relocating the line item. It appears only on data and code lines, so that the descriptor line may be identified by the absence of this item. 0 means an absolute item. 1 through 4 indicate one of the four control sections used by the compiler. A number greater than four indicates an external reference to a run-time support routine. Negative numbers are forward references.

(5) Code lines are printed in MACROSYMBOL format. Again, addresses are relative to the specified control section, except when the control section is 1 (code), when the address is absolute.

(6) These three instructions are used to translate the condition code setting into a 0 or a 1 in a register. In this case, a numeric truth value is not required and the condition code may be used directly for the branch. As a result the compiler 'backs up'. This is indicated by the addresses of the generated code.

(7) This branch is relative to a forward reference. The destination of the branch is the end of the while statement. The forward reference will be defined when the corresponding end is encountered.

(8) Call to the external routine DEC, which performs binary-to-decimal conversion.

(9) This is a descriptor for the string ' '. The first byte contains the length and the last three bytes contain the byte address of the first

byte, relative to the string control section.

(10) Calls to the CONCAT routine that performs the concatenation. Note the way in which the operands are stored. This is responsible for the limitation on nested calls to CONCAT. In addition, if compactification takes place within a call sequence, operands will be changed.

(11) Call to the OUTPUT routine. Note how the implicit parameter for unit 0 is supplied.

(12) This is the definition of the forward reference mentioned in (7).

(13) This is the other way of using the $-toggles.

(14) Standard instruction that terminates all XPL programs.

(15) The forward reference being defined here is the value of NDESCRIPT.

(16) Note that all addresses are absolute in this end-of-program symbol dump.

```
JOB          VERSION C05M
APL L,C,O  [1]

PL COMPILATION - UNIVERSITY OF WASHINGTON - RELEASE A00

 1    DECLARE B BIT(8)  INITIAL(8)*                    7  [2]          01 [3]   0 [4]
 2            H BIT(16) INITIAL(2)*                    8          0002         0
 3            F FIXED   INITIAL(3)*                    9      00000003         0
 4            C CHARACTER INITIAL (*)IS THE VALUE OF F*;1      11000000
 5    DO WHILE F < 4;                                  1A0E LW,R [5]      9       2
                                                       1A0F CI,R          4       0
                                                       1A10 S,1          E0       0
                                                       1A11 BCR,1      1A13       1
                                                       1A12 LI,1          1       0
                                                       1A10 BCR,1         0      -2 [7]
 6        F = F + 1;                                   1A11 LW,R          9       2
                                                       1A12 AI,R          1       0
                                                       1A13 STW,R         9       2
 7        OUTPUT = F || * * || C;                      1A14 LW,R          9       2
                                                       1A15 STW,R        -2      17
                                                       1A16 BAL,O         0      17
                                                       1A17 STW,O        -2      15
                                                         2      01000011
                                                       1A18 LW,R          2       3
                                                       1A19 STW,R        -3      15
                                                       1A1A BAL,O         0      15
                                                       1A1B STW,O        -2      15
                                                       1A1C LW,R          1       3
                                                       1A1D STW,R        -3      15
                                                       1A1E BAL,O         0      15
                                                       1A1F LI,R          0       0
                                                       1A20 STW,R        -2      12
                                                       1A21 STW,O        -3      12
                                                       1A22 BAL,O         0      12
```

[6]

[8]

[9]

[10]

[11]

48

92-12-06

```
 8    END;

 9    /* SYMBOL DUMP */ [13]
10    EOF

IC CARDS CONTAINING 4 STATEMENTS WERE COMPILED.
NO ERRORS WERE DETECTED.

CONTROL SECTION BASES:
     1 1A0E
     2 1A26
     3 1A3C
     4 1A3*

SYMBOL  TABLE  DUMP

IDENTIFIER                TYPE                    LOC'N CSECT LINE REFS [16]

                          BIT(8)                  1A2D    2    1    8
B                         CHARACTER               1A31    3    4    1
C                         FIXED                   1A2F    2    3    4
F                         BIT(16)                 1A2E    2    2    8
H

!CLOAD
!ROOT (FILE,FT,[redacted]FILE,SP,MTS)
[redacted]
LOADING WAS COMPLETED
!PRV
4 IS THE VALUE OF F
!FIN
```

1A23 HCR#0 1A04 1 [12]
1A2* -2 1
1A2A CALLF 1 -1 1 [14]
0 [15]

APPENDIX 6

There are no standard mnemonics for the XDS Sigma standard object language. [4, Appendix A] The mnemonics used on the XPL compiler are listed here as an aid to interpretation.

OBJECT LANGUAGE MNEMONICS

Mnemonic:	Meaning:
AC	Add Constant
AVD	Add Value of Declaration
DEF	Declare External Definition Name
DEFD	External Definition
EOM	Module End
FRD	Forward Reference Definition
LA	Load Absolute
LRL	Load Relocatable (Long Form)
LRSD	Load Relocatable Relative to Declaration
LWS	Load Word Absolute (LA, 4 bytes)
NSCS	Declare Nonstandard Control Section
ORG	Origin
REF	Declare Primary External Reference Name
RL	Repeat Load
XEND	Expression End

REFERENCES

1. McKeeman, William M., Horning, James J., and Wortman, David B. A Compiler Generator Englewood Cliffs, N.J.: Prentice-Hall (1970)

2. Xerox Data Systems XDS Sigma 5 Computer Reference Manual Publication 90 09 59D

3. Xerox Data Systems XDS Sigma 5/7 Macrosymbol Reference Manual Publication 90 15 78A

4. Xerox Data Systems Real-Time Batch Monitor (RBM) Reference Manual Publications 90 15 81C

5. DeRemer, Franklin L., "Simple LR(k) Grammars" C.A.C.M. 14 #7 (1971) pps. 460-467